

Maximizing performance of object detection task using human in the loop machine learning

Venkata Gangadhar Sriram, Peddireddy(11374187)
IFN704 Assessment 3

1.0 Executive Summary:

Object detection is very important machine learning task. Object detection has been around for more than 20 years[2]. Object detection algorithms are trained to identify the objects and their position in an image. There are several algorithms developed for object detection in the past 2 decades. There are several variations of these algorithms, and several pipelines developed to maximize accuracy and speed of these algorithms. One of the best models in the market know for its accuracy and speed in the YOLO object detection algorithm[3]. YOLO models have high accuracy, and they are also fast. Some of the top performing models on benchmarks like COCO-O are YOLO models or variations of a YOLO model. There are several variations of YOLO models like YOLOX[4] and PP-YOLOE[5] models built to improve performance. Despite having very high performance, YOLO or any other model do not compare to the human ability to identify and detect object location. In this project, the performance of the YOLO model is maximized by using a specialized pipeline that leverages the human ability to detect objects and integrates it with YOLO. This model is compared with two other models to evaluate the performance of the YOLO model trained using a specialized pipeline.

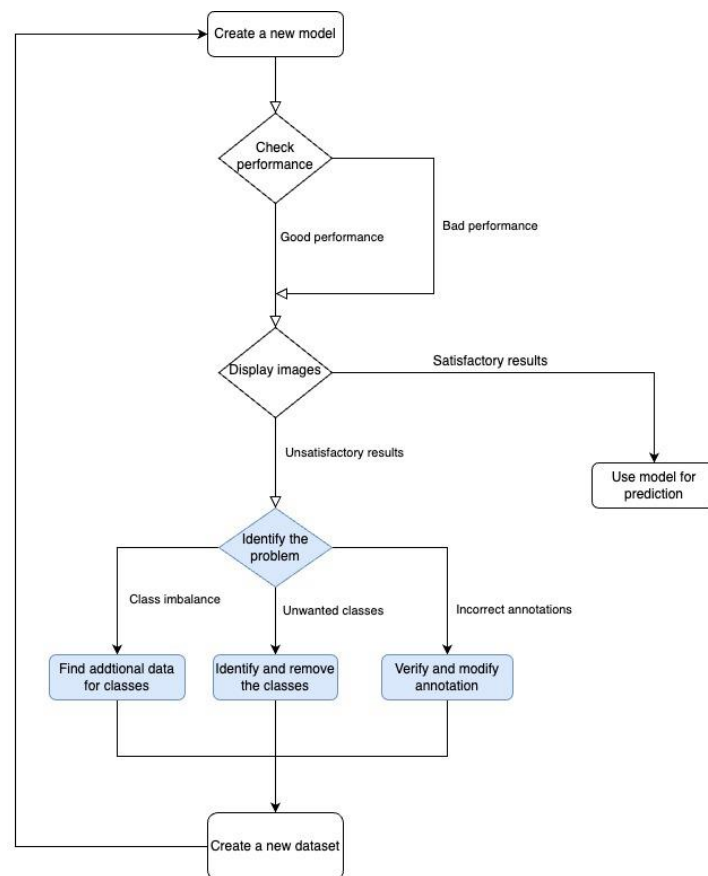
2.0 Introduction:

Object detection is a very important and basic task in computer vision, enabling applications from self-driving cars to real-time surveillance systems. It teaches the computer to identify and locate objects within an image or video, requiring both classification and precise spatial information. Object detection task have been around for more than 20 years. With advancements in deep learning, modern object detection algorithms utilize neural networks to automatically learn high-level features, this vastly improved performance.

One such algorithm developed is You Only Look Once (YOLO) algorithm. The You Only Look Once (YOLO) algorithm has significantly influenced object detection due to its unique approach to real-time detection. Unlike previous algorithms, YOLO treats object detection as a single regression problem, processing the image in one pass rather than scanning multiple regions individually. YOLO views object detection as a regression problem to spatially separated bounding boxes and associated class probabilities. This design makes YOLO exceptionally fast and suitable for real-time applications where speed is critical. YOLO has model has been updated overtime with several version of YOLO being developed. Each version of YOLO brings more features and better performance overtime. YOLO is also one of the best model for object detection around with high performance and great usability.

Despite these advancements, object detection models are not immune to errors, especially when facing novel or complex environments. Humans ability to identify and track objects is exceptional. If a machine learning model can be trained by leveraging human intelligence, this would significantly improve the performance of the model. This is where Human-in-the-Loop (HITL) Machine Learning becomes essential[6]. HITL allows a model to leverage human expertise for correcting misclassifications and improving detection quality. By incorporating human feedback, models can iteratively learn from errors and adapt to new patterns in data, leading to more robust performance in diverse scenarios. This interactive learning process also allows for continuous improvement without the need for complete retraining, optimizing both accuracy and efficiency.

This experiment is designed to leverage user feedback to maximize the performance of a standard object detection algorithm. The object detection algorithm starts with human knowledge. The images are initially annotated by a human before training validation and testing. This project uses a pre-annotated dataset from roboflow website. This dataset is Car-detection v-2. This project looks at maximizing the performance of the object detection algorithm by modifying the dataset using user feedback. The model is trained on different version of data and the data is modified based on user feedback and the model is retrained from scratch and the performance is noted. This pipeline is displayed below.



The boxes that are blue are the steps where with user feedback was used. In this project the user feedback is the feedback from the developer. The user is required to identify the error in the dataset, Find additional data, identify and remove the class, and verify and modify annotations.

In this experiment 3 models are built/trained on 3 different version of data. The first version of the data was the full original data. The second version of the data is built with minimal user intervention. The third version of the data is built using all the knowledge acquired and a lot of user intervention. The modification made to the data and the result are discussed further below. This experiment aims to create a superior model with good performance by integrating user feedback.

3.0 Literature Review:

Object detection has been a rapidly evolving field in computer vision, focusing on identifying and localizing objects within images. Early approaches to object detection often relied on manually engineered features, such as Haar-like features and Histograms of Oriented Gradients (HOG), which were fed into traditional machine learning classifiers. These methods were limited in scalability and struggled with complex, real-world scenes due to their inability to adaptively complex features and complex relations. With the emergence of Convolutional Neural Networks (CNNs), object detection achieved significant improvements in both speed and accuracy, especially through the development of region-based algorithms like R-CNN and Fast R-CNN[7]. These region-based detectors introduced the idea of using selective search to identify object-like regions before classification, which led to better performance but was computationally expensive, particularly for real-time applications.

The You Only Look Once (YOLO) algorithm[3], revolutionized object detection by reimagining the detection process. Unlike traditional methods that used multiple stages to identify regions and classify them, YOLO treated object detection as a single regression problem. By dividing the image into a grid and predicting bounding boxes and class probabilities directly for each grid cell, YOLO achieved detection in a single pass, significantly improving inference speed. YOLO was processing 1 image at-a-time this made YOLO significantly faster than its predecessors.

Since YOLO's initial introduction, successive versions have improved the model's accuracy and adaptability. For instance, YOLOv3 introduced residual blocks and multi-scale predictions, enhancing detection across varying object sizes. YOLOv5 and YOLOv8 have incorporated advanced techniques like Anchor-free detection and self-adaptive methods for data augmentation, which further improve performance and ease of use. These enhancements make YOLO particularly versatile for a wide range of applications, from autonomous vehicles to surveillance, where real-time processing is essential.

Despite advances in algorithmic accuracy, object detection models are still challenged by complex and dynamic environments, leading to misclassifications or localization errors. This is especially true when encountering scenarios underrepresented in the training data. Human-in-the-Loop (HITL) machine learning is an interactive approach designed to address these issues by involving human experts to correct model predictions and guide the learning process. Studies have shown that integrating human feedback can enhance model adaptability and reliability, which is crucial for applications requiring high accuracy, such as medical imaging or autonomous systems.

In object detection, HITL can assist in two primary areas: error correction and model updating. By reviewing misclassified or incorrectly localized objects, human annotators can provide corrections that are fed back into the model for fine-tuning. Techniques such as active learning allow models to query human experts for annotations on uncertain predictions, ensuring the model focuses on learning from its weaknesses. Combining HITL with advanced algorithms like YOLO creates a robust system capable of handling high-variance environments by continuously adapting to real-world conditions through human guidance.

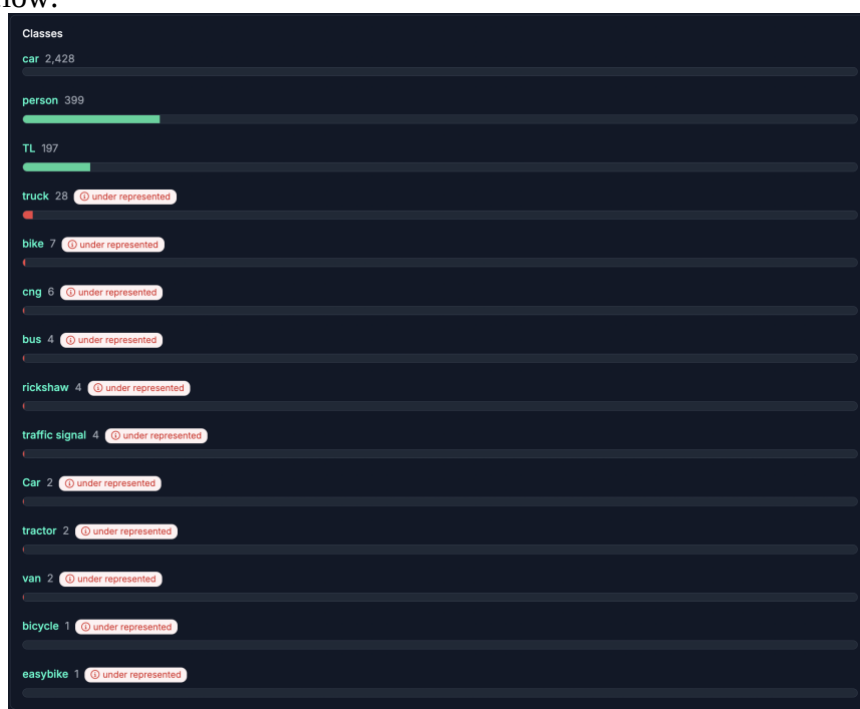
The evolution of object detection has been driven by both algorithmic innovations, like YOLO's shift to single-stage detection, and methodological advancements in model training, including HITL frameworks. Together, these developments represent a comprehensive approach to creating adaptable and accurate object detection systems that perform reliably even in complex scenarios. Integrating YOLO with HITL offers a powerful solution for modern applications, enabling real-time detection with continuous improvement through human oversight.

4.0 Methodology:

4.1 Data Preprocessing and data modifications:

As mentioned in section 2.0 this project compares 3 models. The three models use 3 different versions of the dataset.

The original dataset was obtained from roboflow and the all the images in the dataset were annotated. This dataset has 14 classes. The first model utilizes all the data that has been provided without any modifications. The specifications of the original dataset are displayed in the image below.



4.1 The number of instances of each class present in the dataset.

This version of the dataset has 1121 training images, 362 validation images and 282 test images. Absolutely no preprocessing has been done on this dataset.

The second model is built a modified version of the dataset. There is a dummy class 'Car' has been removed. The instances present in the class 'Car' are also present in the class 'car' making it a duplicate class i.e., these images are annotated twice. Majority of the remaining classes are removed or merged with other classes. All classes with less than 10 instances have been visually inspected, these classes are merged with other classes, or they have been removed. Upon visual inspection and analysing the results obtained by the full model the classes 'easybike', 'rickshaw' and 'cng' has been removed. The classes 'van', 'bicycle' and 'traffic signal' are merged with classes 'car', 'bike' and 'TL'. These classes have a very small number of images and removing them would improve performance. However, these classes

like ‘van’ and ‘bike’ are relevant classes important for objective of the algorithm. The objective of the algorithm is to being able to identify the major types of classes of vehicles while also being able identify people in traffic conditions.

The dataset used for the final model had a lot of modification made to maximize the performance. Despite merging and eliminating classes there was still class imbalance in the dataset. The dataset is balanced by merging more classes together and adding additional data to the dataset. To further improve the performance, the class ‘bike’ is merged with the class ‘person’. This is done by visually inspecting all the images in the ‘bike’ class. All the images where the bike is not present with a person actively operating it on the road have been removed. The reasoning behind removing merging the two class was, in majority of traffic conditions there is a person operating the bike. Therefore, identifying the person on the bike could also be useful feature on the algorithm removing the class bike would also reduce the strain on the algorithm. In an ideal scenario, person and bike are identified as two different entitles. However, this would require a massive amount of data where there are datapoints with bikes and people with several images, where there is a good amount of intersection between these bike and people classes and other classes. Finding and annotating this dataset would be tedious and this dataset would add additional complexity to the algorithm.

Additional images from external datasets were added to the classes ‘bus’ and ‘truck’. These datasets were selected from roboflow, with 2 main criteria. The images in these datasets would properly identify the classes and there should be images with these classes present in the traffic conditions. Based on the results of the previous model the classes car, bus and truck needed expanding. To get the best results 3 datasets are chosen, where each dataset only had images that highlighted one of the 3 classes mentioned. For example, a dataset was chosen where only the class ‘bus’ was highlighted. These datasets were merged with the original dataset in roboflow. All the images added were then inspected and all the classes that were not annotated in this new dataset were annotated. Finally, the class ‘TL’ where all the traffic lights were highlighted were removed. This algorithm prioritized identifying vehicles on the road in traffic conditions. The specifications of the final dataset are displayed below. For this dataset data augmentation was applied to increase the number of images. Only one augmentation is applied where the images were flipped horizontally.

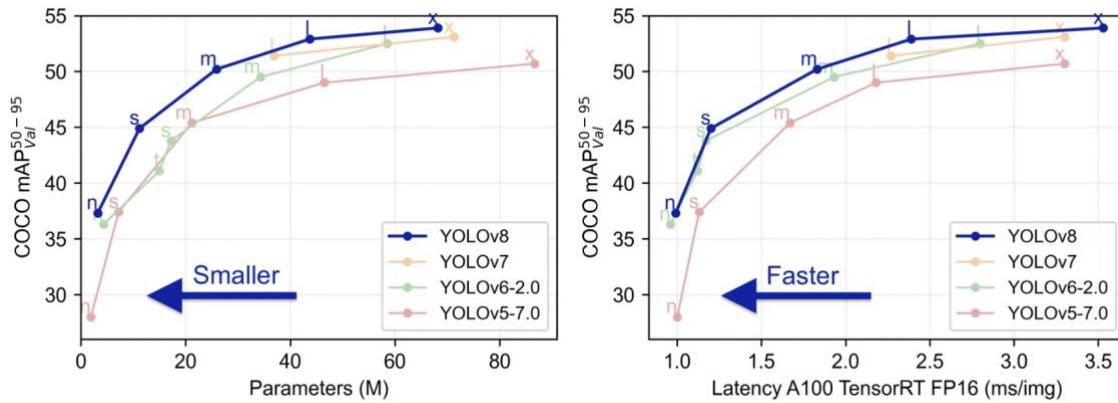
Class	Number of annotations
car	3304
bus	590
truck	513
person	628

In second and third datasets the class car also includes vans and pickup trucks. The class truck is classified as any vehicle which has a large back carriage which can be used for transporting goods in a bulk. The images in the original dataset were 192x192 and the images added to the dataset have varied resolutions. While training the images were resized to 640x640.

4.2 Models:

This project prioritizes the improving the performance of the model by improving the data quality by using the feedback of the user. The base model trained with all 3 versions of the data is YOLOv8m.

This project is specifically run on python. To make the coding section easier the version chosen was a version after YOLOv5. These versions were designed with pytorch library. The relative performance of YOLO models from YOLOv5 to YOLOv8, on the COCO are displayed below.



The graph to the left is plotted between parameters and the performance. As the graph displays YOLOv8 series has achieved better performance than all the other YOLO series with a smaller number of parameters. This implies training any model in YOLOv8 series, with data, is significantly easier than training any other YOLO series. The graph to the right is plotted between a measure of performance(mAP) and a measure of speed of the model under a specific condition. It shows that YOLOv8 is faster while retaining a better performance.

Model	size (pixels)	mAP ^{val} ₅₀₋₉₅	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

YOLOv8m is specifically chosen for this project. As shown in the table above YOLOv8m has a relatively small increase in the performance compared to YOLOv8l despite having almost 18 million more parameters.

As for the hyperparameters used, all three models have been trained for 30 epochs.

4.3 Parameters for model evaluation:

This model evaluates all the results obtained from the model such as precision, recall, mAP50 and mAP50-95. All three models presented in this project are part of the pipeline to maximize performance using user feedback. The model performance is evaluated on the validation set. The validation set of the models are completely different as these models are trained on different version of data as mentioned earlier. However, the performance of these models can be evaluated based on their ability to identify different classes present. The main observation would be the gradual increase of performance of the models as the data is modified and perfected.

The precision of the model is similar to the precision observed in a classification problem. This value gives the percentage of the instances in a class that have been classified correctly. It is ratio between the true positive instances and the total instances detected in a class. Higher precision would mean that there are fewer false positives.

The recall is also similar to recall in the classification problem. This value gives the percentage of the instances that are detected in a specific class. This value is a ratio between true positives instance and all the instances in the class originally. Higher recall would indicate that the model is very good at detecting instances in the class.

Mean Average Precision is a unique value widely used to measure the performance of object detection models. Mean Average Precision evaluates the precision of a class with an Intersection over Union (IoU) factor within a certain range. Two different mAP values are used in this experiment, mAP50 and mAP50-95. For mAP50, this threshold means the bounding boxes predicted by the algorithm should overlap with more than 50 percent of the predicted boxes of the ground truth. mAP50 gives a sense of the model accuracy. mAP 50-95 calculates the average of the mean average precision for threshold 0.5 to 0.95 with increments of 0.05. This value gives an estimate of the model's localization capabilities.

The trained models are deployed and tested. The trained models are saved. The models are then used to predict the validation set images. These images are then displayed. The validation set for all three version of the dataset have more than 300 images. Displaying and analysing all the images would be tedious. Only 20-30 images are selected displayed and analysed to understand the capabilities of the dataset. This can be useful in scenarios where the mAP50 and mAP50-95 are very low for the whole model or just a single task.

5.0 Results and evaluation:

Each model trained on a specific set of data produces results obtained from the validation set towards the end. Some images with the bounding boxes predicted by the results are displayed

5.1 Model 1 results:

Displayed below are the results obtained from the validation set after training. The classes have been removed from the image because the classes have been jumbled during training.

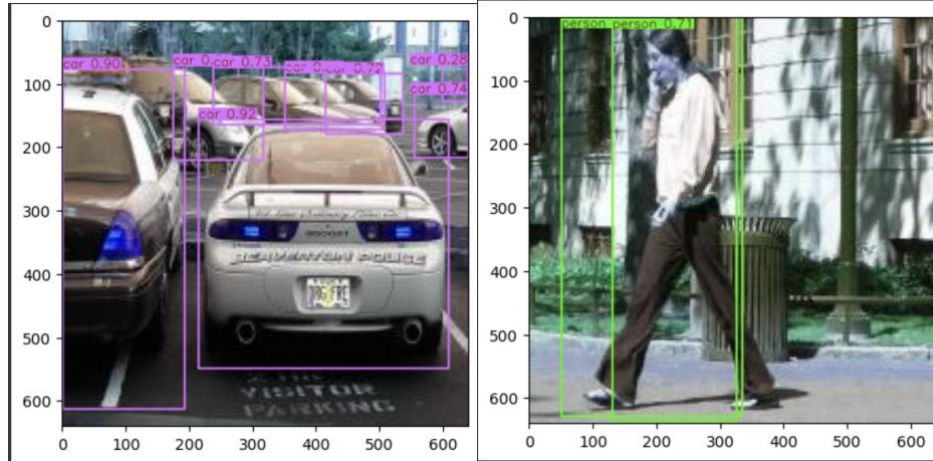
Images	Instances	Box(P	R	mAP50	mAP50-95) :
362	657	0.776	0.309	0.337	0.219
17	40	0.787	0.4	0.49	0.278
3	3	1	0	0	0
313	518	0.85	0.875	0.918	0.693
1	3	1	0	0	0
38	73	0.782	0.785	0.804	0.479
1	1	1	0	0	0
1	2	0	0	0	0
11	17	0.786	0.412	0.48	0.303

There are only 8 classes displayed out of the 14 classes present in the dataset. The classes displayed at the end are the only classes detected by the model in the validation set dataset. There are 2 class with a 1 instance each and 3 classes with only 2 instances. These classes lack enough data to be present in all 3 partitions of the dataset. This would mean some of these classes would not have data to train or data to validate these findings. This severe lack in the data available would explain the under performance of classes like 'easybike', 'rickshaw' and 'bike'.

According to the displayed results only 30.9 percent of the instances are detected, and 77.6 percent of these instances were detected accurately, these are indicated by precision and recall. The mAP50 shows that only 33.7 percent of the images have at least 50 percent of the predicted box overlapping with the box in the ground truth. The mAP50-95 score is also low, given the fact that mAP50 is also low, the model is bad at locating the object.

All the classes except 'car', 'person' and 'TL' are underrepresented. The model has completely failed at detecting classes where precision or recall or both precision and recall are 0. The model performance is really good with the class 'car'. The model also performs well with classes 'person' and 'TL'. The model as decent performance with the class 'truck' was also decent. The model failed to recognize other classes.

Two images from the predicted data from the validation set are displayed below.



In the image to the left, it can be observed that the model has decent performance with identifying the class 'car'. The algorithm successfully identified all the instances of the 'car'. The algorithm miss identified background as the class 'car'. This is classic example of wrongly instance identification and leads to increase in false positives and the decrease of precision. A similar example can be seen in the image to the left where an additional annotation the class 'person' has been made.

5.2 Model 2 results:

This model is built with intermediate intervention from the user. The data used for this model is modified slightly with input from user. Upon visual inspection of the class 'cng' and 'easybike' after the anomalous results obtained from the previous model, the instances of these classes are significantly distorted. These instances were difficult to be recognized by the user. An assumption was made that the model is classifying random distortions in the background as instances of these classes. These classes were removed in the next model to observe any improvements in the output. Other modifications suggested in the section 4.1 are also made. Displayed below are the results obtained from the validation set after training. The classes have been removed from the image because the classes have been jumbled during training.

Images	Instances	Box(P	R	mAP50	mAP50-95):
362	652	0.772	0.348	0.4	0.254
17	40	0.823	0.55	0.581	0.327
2	2	1	0	0	0
313	518	0.906	0.837	0.925	0.698
38	73	0.901	0.699	0.803	0.453
1	2	0	0	0	0
11	17	1	0	0.0938	0.0445

As mentioned earlier the model displayed only 6 classes, despite 8 classes being used, is because the model only detected 6 classes. Despite rebalancing the dataset there is severe class imbalance observed in several classes. The model does not have enough data to train on most classes.

According to the displayed results only 34.8 percent of the instances are detected, and 77.2 percent of these instances were detected accurately, these are indicated by precision and

recall. There a slight increase in the average number of instances detected per class and a slight decrease in number of instances detected correctly. Overall mAP50 and mAP50-95 has increased. This shows that model 2 is better than model 1 at identifying the general location of objects.

The performance of the model on the classes ‘car’, ‘TL’ and ‘person’ have improved. The performance of the model on truck has dropped from model 1. The mAP50-95 has dropped from mAP50 for all 3 classes mentioned this means that the model has problem localizing the instances. The model has trouble detecting other classes.

Two images from the predicted data from the validation set are displayed below.



The image on the left shows that the model has trouble identifying the class TL and the model is also picking up an instance of background noise present. In this image we can see an example of a false negative being picked up by the image. The image to the left shows that model has no problem picking up multiple instances in an image.

5.3 Model 3 results:

This model is built with a lot of data which has taken input from the user or the developer. The data used in the model is a modified and further expanded version of the previous dataset. The classes ‘bike’, ‘tractor’, ‘bus’ and ‘truck’. The modification suggested in section 4.1 are applied and the ‘bike’ class is merged with the class ‘person’. Other modifications and expansions to the dataset mentioned in section 4.1 were applied to the dataset. These modifications are applied to classes ‘truck’ and ‘bus’ classes only. The class ‘tractor’ is removed for a lack data available and the lack of the viable datasets. The class ‘car’ is further expanded to improve the accuracy of the class further. Finally, the class ‘TL’ has been removed as this reduces the burden on the model. Despite the removal of the class ‘TL’ the model still maintains its utility by still retaining the ability to identify major vehicle classes and people. Displayed below are the results obtained from the validation set after training.

Images	Instances	Box(P	R	mAP50	mAP50-95):
673	1128	0.893	0.748	0.825	0.648
146	204	0.928	0.75	0.812	0.731
448	699	0.941	0.85	0.922	0.723
71	122	0.787	0.575	0.677	0.39
94	103	0.914	0.816	0.892	0.749

All 4 classes are displayed. This means that all 4 classes are detected by the model.

According to the displayed results 74.8 percent of the instances are detected, and 89.2 percent of these instances were detected accurately, these are indicated by precision and recall.

There a massive increase in the average number of instances detected per class and an increase in number of instances detected correctly. Overall mAP50 and mAP50-95 have also been increased (more than doubled). This shows that model 3 has is far superior at localization. The dip in mAP50-95 when compared to mAP50 show that this model still has issues with localization in certain instances.

The model performance is relatively low on the class 'person'. This would performance dip can be interpreted such that the model has problem localizing the class 'person'.

Two images from the predicted data from the validation set are displayed below.



The image has an instance of the class 'person' labelled twice. The other instance was labelled correctly. The second image on the left has an image instance labelled twice both with the wrong instance class.

The results on the test set for the final model.

Images	Instances	Box(P	R	mAP50	mAP50-95):
403	687	0.905	0.818	0.883	0.688
42	55	0.916	0.79	0.84	0.757
260	404	0.902	0.856	0.885	0.676
84	170	0.896	0.763	0.876	0.498
47	58	0.908	0.862	0.931	0.82

The results are similar to the results on the test set. This is model has high performance on all class however the model problem with localization especially with the class 'person'.

6.0 Conclusion:

The 3 models built for this project are part of the pipeline to maximize the performance of the model at detecting vehicle classes. The models 1 and 2 are the models that have built during the trail phase of this experiment. As mentioned earlier this experiment aims to maximize the performance primarily using human feedback. The significant increase in the performance proves that incorporating human knowledge into machine learning training process helps improve the performance of the model.

The final model built with feedback of the user (the developer in this case) is very effective at identifying and locating cars, trucks, buses and people. The class car also encompasses minivans vans and pickup trucks. The class people also include people walking in traffic and people operating motorbikes or bicycles in traffic conditions. The model can identify and locate several major types of vehicles.

Despite, the model having good performance, this model has several short comings. The model cannot recognize the all the classes removed from the dataset during the development, like 'tractor', 'easybike' and 'cng'. The model will not be able to recognize all

the vehicles present on the road. These instances are not labelled so the model will ignore these instances, or the model could misclassify these instances. Certain classes of the vehicles are combined to obtain a better result. This model would be useful for identifying and locating all the instances present in an image of a traffic scenario in a very limited number of cases. The model might get most instances but not all instances.

This model cannot be deployed for a real time traffic safety scenario because of two reasons. The model does not cover all the vehicle classes, and the model has not achieved enough performance.

An ideal algorithm for traffic safety scenario would be trained on all the classes of vehicles with a well-balanced dataset.

This project utilized the capabilities of a single user (the developer). This project can be further expanded by using a larger dataset with more classes and involving multiple users in dataset collection, annotation and verification. This would allow for a more sophisticated model being developed.

7.0 References:

- [1] H. Silyn-Roberts, *Writing for science and engineering: papers, presentations, and reports*, Second edition. in Elsevier insights. Amsterdam: Elsevier, 2013.
- [2] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, 'Object Detection in 20 Years: A Survey', *Proc. IEEE*, vol. 111, no. 3, pp. 257–276, Mar. 2023, doi: 10.1109/JPROC.2023.3238524.
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, 'You Only Look Once: Unified, Real-Time Object Detection', in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2016, pp. 779–788. doi: 10.1109/CVPR.2016.91.
- [4] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, 'YOLOX: Exceeding YOLO Series in 2021', arXiv.org. Accessed: Nov. 03, 2024. [Online]. Available: <https://arxiv.org/abs/2107.08430v2>
- [5] S. Xu *et al.*, 'PP-YOLOE: An evolved version of YOLO', arXiv.org. Accessed: Nov. 03, 2024. [Online]. Available: <https://arxiv.org/abs/2203.16250v3>
- [6] S. Budd, E. C. Robinson, and B. Kainz, 'A survey on active learning and human-in-the-loop deep learning for medical image analysis', *Med. Image Anal.*, vol. 71, p. 102062, Jul. 2021, doi: 10.1016/j.media.2021.102062.
- [7] R. Girshick, 'Fast R-CNN', in *2015 IEEE International Conference on Computer Vision (ICCV)*, Dec. 2015, pp. 1440–1448. doi: 10.1109/ICCV.2015.169.